CSCI2000U MIDTERM - SIDE A: EXAM TRAPS, UNIX, GIT

EXAM TIPS (Fri Nov 6)

MCQ + short + long answers. Concepts matter more than syntax. Trace code line by line.

NOT tested: git log --pretty, Jupyter shortcuts, SSH setup.

PROF'S A-OR-B TRAPS

`cp -r adir/ cdir` copies the CONTENTS into cdir. `cp -r adir cdir` puts adir INSIDE cdir.

`print(a,b,sep="_",end="!")`: sep goes BETWEEN items, end goes AFTER the line (no newline).

`"banana".count("ana")` = 1: matches never overlap.

No `return` gives `None`. `lst.sort()` returns `None`.

Slices, `reshape`, `ravel` = VIEWS. `copy`, `flatten` = copies.

`.gitignore` does NOT untrack a file. `restore f` destroys edits. `revert` ADDS a commit.

`axis=0` = one answer per COLUMN, `axis=1` = per ROW.

`open(f,"w")` ERASES the file. `"a"` appends.

UNIX BASICS (L01-L03)

Bell Labs, Thompson + **Ritchie**, written in **C**. **Linux = Unix-like**, not UNIX. macOS is UNIX.

Terminal = window. **Shell** = runs commands (bash, zsh). **Prompt** = `$`. **CLI** = text interface.

Format: `command -options arguments`. Case-sensitive. Tree root is `/`.

PATHS (L02)

Absolute starts with `/` or `~`. **Relative** does not.

`~` home, `.` here, `..` parent, `cd -` previous dir. `..` at `/` stays at `/`.

Trace: from `/home/sam/docs`, `cd ../data` ends at `/home/sam/data`.

Hidden files start with `.` (`ls -a`).

TUTORIAL TE2 (SHELL)

`mkdir project{1..3}`	project1 project2 project3
`touch notes{01..03}.md`	notes01.md ... notes03.md
`mv ~/lab2/*.md docs/`	all .md files into docs/
`chmod a=r f` / `u+x f` / `640 f`	all read-only / owner +x / rw-r-----

SHELL COMMANDS (L03)

`pwd`	print working directory
`cd dir cd .. cd ~ cd -`	into dir / up / home / previous
`ls`	`-l` long, `-a` hidden, `-S` by size, `-r` reverse, `-R` recursive. `ls -RSral` = all of these. `ls main*` = wildcard
`mkdir d`	`-p a/b/c` makes parents. `mkdir p{1..3}` = p1 p2 p3
`touch f`	create an empty file
`echo "x" > f`	**overwrite** f. `>> f` **appends**
`cat head tail`	show file / first lines / last lines
`man cmd which cmd`	manual page / where a command lives

FILES (L04)

`cp a b`	copy. `cp -r dir new` copies a directory
`mv a b`	move OR rename. Into an existing dir = moves inside
`rm f`	delete (no undo). `rm -r dir` deletes dir + contents

PERMISSIONS (L03)

`ls -l`: `-rwxr-x--x` = type (`-` file, `d` dir), then owner, group, others.

r=4 w=2 x=1. **755**=rwxr-xr-x, **644**=rw-r--r--, **640**=rw-r-----, **712**=rwx--x-w-.

Symbolic: `chmod u+x f`, `chmod g+r *.txt`, `chmod a=r f`, `chmod u=rw,go= f`.

A directory needs `x` to `cd` into it.

VERSION CONTROL

VCS beats a backup: history, who/why, restore any version, collaborate. Git is **distributed** (every clone has full history). Commits are **snapshots**.

3 areas: working dir -> staging area -> repository (`.git`).

4 states: untracked, modified, staged, unmodified.

GIT COMMANDS (L06-L07)

`git init`	folder becomes a repo. `git clone url` copies a remote
`git status`	untracked / modified / staged?
`git add f`	stage f. `git add .` stages everything
`git commit -m "msg"`	save the staged snapshot
`git log`	newest first. `--oneline`, `-n 3`
`git show HEAD`	one commit. `HEAD~1` = parent, `HEAD~1:f` = f then
`git diff`	working copy vs **staged**
`git diff --staged`	staged vs **last commit**
`git rm f`	delete from disk + stage it
`git rm --cached f`	stop tracking, KEEP the file

UNDO (L06-L07)

`git restore --staged f`	**unstage**. Edits stay in the file
`git restore f`	**discard** edits. Permanent
`git checkout HEAD f`	old way to restore f from HEAD
`git revert --no-edit HEAD`	NEW commit that undoes HEAD (log 3 -> 4)

STAGED + MODIFIED

`git add n.txt` then edit n.txt again: `git status` shows it **staged AND modified**. `add` stages the file AS IT IS at that moment; later edits need another `add`.

.GITIGNORE

Lists patterns (`*.log`, `data/`). Affects **untracked** files only.

Committed `secret.env`: `git rm --cached secret.env`, add to `.gitignore`, `git add .gitignore`, `git commit`.

GitHub gets your **public** key, never the private.

TUTORIAL TE3/TE4 (GIT)

`add` then `restore --staged f`: back to **modified**, edits kept.

Commit + edit: `git diff` shows it. After `add`: `git diff` EMPTY, `git diff --staged` shows it.

`git show HEAD~1:readme.md` = file in the previous commit.

CSCI2000U MIDTERM - SIDE B: PYTHON, DATASTRUCTURES, NUMPY, FILES

PYTHON BASICS (L08)

Expression gives a value (`3+4`). **Statement** does something (`x=5`, `if`, `print()`). Jupyter shows only a cell's LAST expression.

Types: `int float str bool`. `type(x)`, `int("5")`, `float("2.5")`, `str(5)`.

`/` float, `//` floor, `%` remainder, `**` power: `7/2`=3.5 `7//2`=3 `7%2`=1 `2**3`=8. `=` assigns, `==` compares.

`math.sqrt(144)`=12.0, `math.isqrt(144)`=12, `round(3.14159,3)`=3.142.

Dynamic typing: `x=5` then `x="hi"` is fine. **Strong typing:** `"5"+3` is a TypeError.

STRINGS (L10)

Index from 0, `s[-1]` last. `s[a:b]` stops BEFORE b. `s="data_science"`: `s[5:]`="science", `s[::-1]` reversed, `s[::2]` every 2nd.

Immutable: `s[0]="J"` is a TypeError. `s.upper()` returns a NEW string.

`+` join, `*` repeat, `len`, `in`, `.lower()` `.strip()` `.replace(a,b)` `.split(",")` `.find(x)` `.count(x)`, `",".join(lst)`.

f-string: `f"{name} scored {x:.2f}"`.

CONTROL FLOW (L09)

`if/elif/else` stops at the FIRST true branch. Separate `if`s ALL run. Ternary: `"pos" if x>0 else "neg"`.

`range(n)` = 0..n-1. `range(1,10,3)` = 1 4 7. `enumerate(lst)` gives (index, value). A `while` must change its condition.

Comprehension: `[v for v in vals if v>7]`.

```
def is_leap(y):
    return (y%4==0 and
      (y%100!=0 or y%400==0))
```

TUTORIAL TE4/TE5 OUTPUTS

`b=a.copy(); b.append(4)`	a=[1,2,3] b=[1,2,3,4]
`nums[::2]`, `nums[::-1]` (1..8)	[1,3,5,7]; [8,7,...,1]
`[i*i for i in range(7)]`	[0,1,4,9,16,25,36]
`power(2,5)` with a loop	32
loop collects `3*i`, 5 items	[3, 6, 9, 12, 15]
`[n**2 for n in range(0,11,2)]`	[0,4,16,36,64,100]

FUNCTIONS (L09)

`def f(a, b=1):` default param. Call by keyword: `f(b=2, a=3)`. **No return** gives `None`.

`*args` = tuple of extras (`f()` gives `()`). `**kwargs` = dict.

`lambda x: 2*x`. `apply_twice(lambda t:2*t, 3)` = 12.

Recursion: base case + a call on a smaller input. No base case = RecursionError.

```
def factorial(n):
    if n == 0: return 1
    return n * factorial(n-1)
# factorial(5) = 120
def add_all(*args):
    return sum(args)
# add_all(1,2,3,4) = 10
```

PRINT & ERRORS (L09)

`print(*objs, sep=" ", end="\n")`. Default `sep` is ONE SPACE (the slide shows `''`).

`print("A","B",sep="_",end="!")` gives `A_B!`.

Traceback: read from the BOTTOM: error type, message, line. `NameError TypeError IndexError KeyError ValueError`.

LISTS & TUPLES (L11)

list `[]` mutable: `append`, `insert(i,x)`, `pop()` removes the LAST, `remove(x)`, `l[-2:]=[30,40]`.

`sort()` in place returns `None`; `sorted(l)` is a new list. `l*3` repeats.

tuple `()` immutable: `t+(4,)` makes a NEW tuple.

Aliasing: `b=a` is the same object. `a.copy()` is SHALLOW (inner lists shared): `a=[1,[2,3]]; b=a; c=a.copy(); a[1][0]=99` changes a, b AND c.

SETS & DICTS (L11)

set `{}` unique, unordered. `set([1,2,2,3])` = {1,2,3}. `a|b` union, `a&b` intersection. `{}` is an empty DICT; empty set is `set()`.

dict `{k:v}`: `d[k]=v` adds OR overwrites, `del d[k]`, `.keys()` `.values()` `.items()`. A repeated key keeps the LAST value.

`{r["name"]:r["score"] for r in recs}`, `[r["name"] for r in recs]`.

NOTEBOOK EXAMPLES (L09-L14)

grade=83: `>80` A, `>70` B, else Fail	A
`print(joined)`, joined is local to the caller	NameError
`"ahahaha".count("aha")`	2
`"banana".find("bana",1,6)`	-1 (not found)
`g[1,2]`; `g[0:2,1]`; `g[:,1]`	92; [59 85]; [59 85 78 88]
`flatten()` edit / `ravel()` edit	original same / original CHANGED

NUMPY (L12-L13)

`import numpy as np`. Arrays: **same type**, **contiguous**. Vectorized = faster in practice, NOT lower Big-O.

`.shape` (tuple), `.ndim` (int), `.dtype`. `np.zeros((2,3))` (a tuple!), `ones`, `full`. `arange(0,6)` ints, `linspace(0,1,5)` floats incl. end. `reshape(2,3)`, `reshape(-1,1)`.

`arr*3` is elementwise (`list*3` repeats). `a[i,j]`, row `a[0,:]`, column `a[:,0]`. **Mask:** `x[x>0]`.

`sum mean min max std`: no axis = one number. `axis=0` one per **column**, `axis=1` one per **row**.

Broadcasting: compare shapes from the RIGHT; OK if equal or one is 1. (3,1)+(4,)=(3,4). (2,3)+(3,)=(2,3). (2,3)+(2,)=ERROR. `np.vstack([R,R])` stacks rows. `np.hstack([R,c])` needs the same row count.

VIEWS vs COPIES (L13)

VIEW (changing it changes the original): slices, `view()`, `reshape()`, `ravel()`.

COPY (independent): `copy()`, `flatten()`. `resize()` changes the array in place; `reshape()` does not.

`b=a[1:3]; b[0]=99` changes `a`.

DATA SCIENCE + FILES (L14)

`with open(path, mode) as f:` **closes the file automatically** at the end. Modes: `r` read, `w` write (**erases**), `a` append.

`csv.writer(f).writerow([...])`. `csv.reader(f)` rows are lists of **strings**. `pd.read_csv("f.csv")` gives a **DataFrame**.

Process: collect -> **wrangle/clean** (>80% of time) -> **EDA** (graphs, summary stats, outliers; NO hypothesis) -> model (explanatory) -> communicate -> product (feedback loop).

TUTORIAL TE6 OUTPUTS

`nums[-2:]=[30,40]`, [1,20,3]	[1, 30, 40]
`a=[1,[2,3]]; b=a; c=a.copy(); a[1][0]=99`	a, b, c ALL [1,[99,3]]
`{1,2,3}|{3,4}`; `&`	{1,2,3,4}; {3}
`row=B[0,:]; row[:]=9`	B row 0 = 9 9 9 (view)
`x[x>0]`, x=[3,-1,5,0,-2]	[3 5]
`vstack([R,R])`, R (3,2)	shape (6,2)